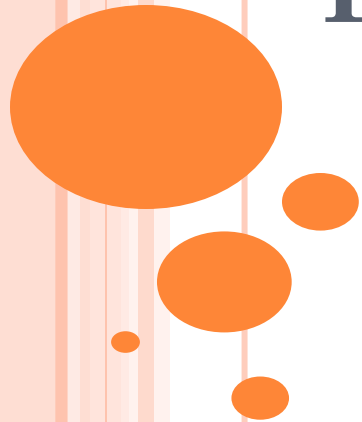


PROGRAMACIÓN REACTIVA EN LA WEB



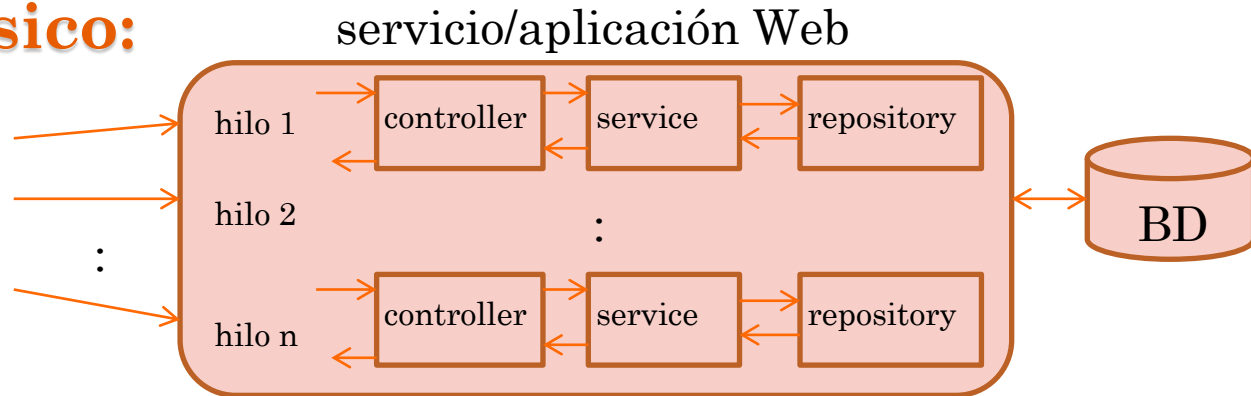
Programación reactiva en Web

➤ Modelo clásico:

(bloqueante)



cada petición un hilo



➤ Modelo reactivo:

(no bloqueante)



Petición=suscripción
(reutilización de hilos)



Microservicios reactivos

- Microservicios implementados según el patrón reactivo.
 - Baja sobrecarga ante un alto volumen de peticiones al poder ser gestionados por menos hilos.
 - El cliente se suscribe a un flujo de datos, que son consumidos según van generándose.
 - Basado en un modelo de programación funcional.
- 

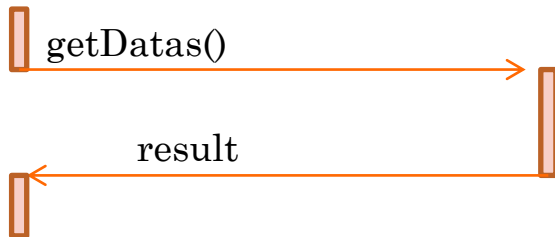
Spring reactor

- **Framework creación de servicios REST reactivos en Spring que utiliza Spring WebFlux en lugar de Spring MVC.**
- **Basado en la generación de flujos reactivos.**
- **Utiliza servidor Netty no bloqueante.**
- **Emplea los siguientes tipos de observables:**
 - **Mono.** Encapsula un dato que es consumido de forma asíncrona
 - **Flux.** Representa un grupo de datos que serán consumidos de forma asíncrona

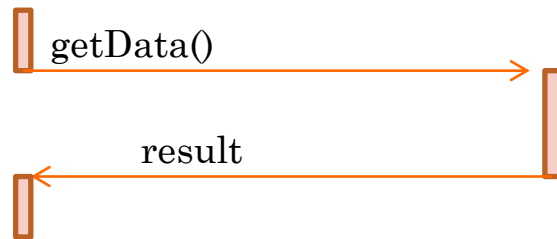
Flujos reactivos vs datos básicos

Llamada estándar
a método

`List<T> getDatas(){..}`

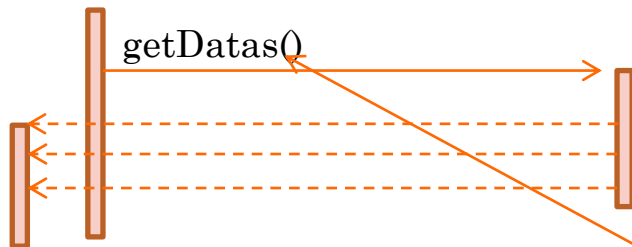


`T getData(){..}`

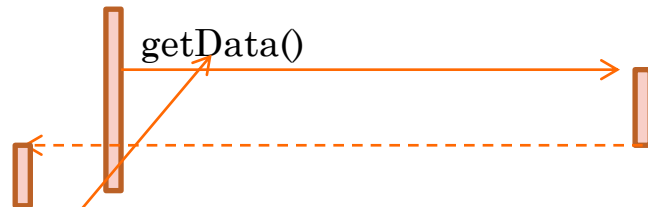


Suscripción a flujo

`Flux<T> getDatas(){..}`

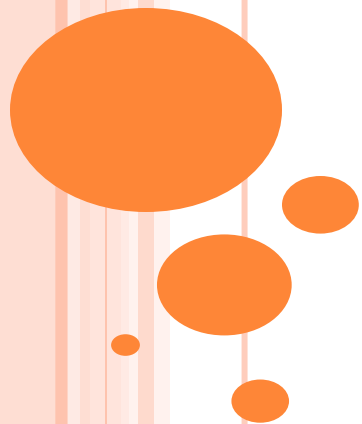


`Mono<T> getData(){..}`



suscripción a flujo, por naturaleza asíncrona

SPRING WEBFLUX



Características

➤ Librería incluida a partir de Spring 5, basada en el Project Reactor para creación de aplicaciones reactivas

➤ Se caracteriza por:

- Utilización de hilos no bloqueantes
- Creación de streams asíncronos no bloqueantes
- Permite a las aplicaciones clientes procesar datos de forma asíncrona



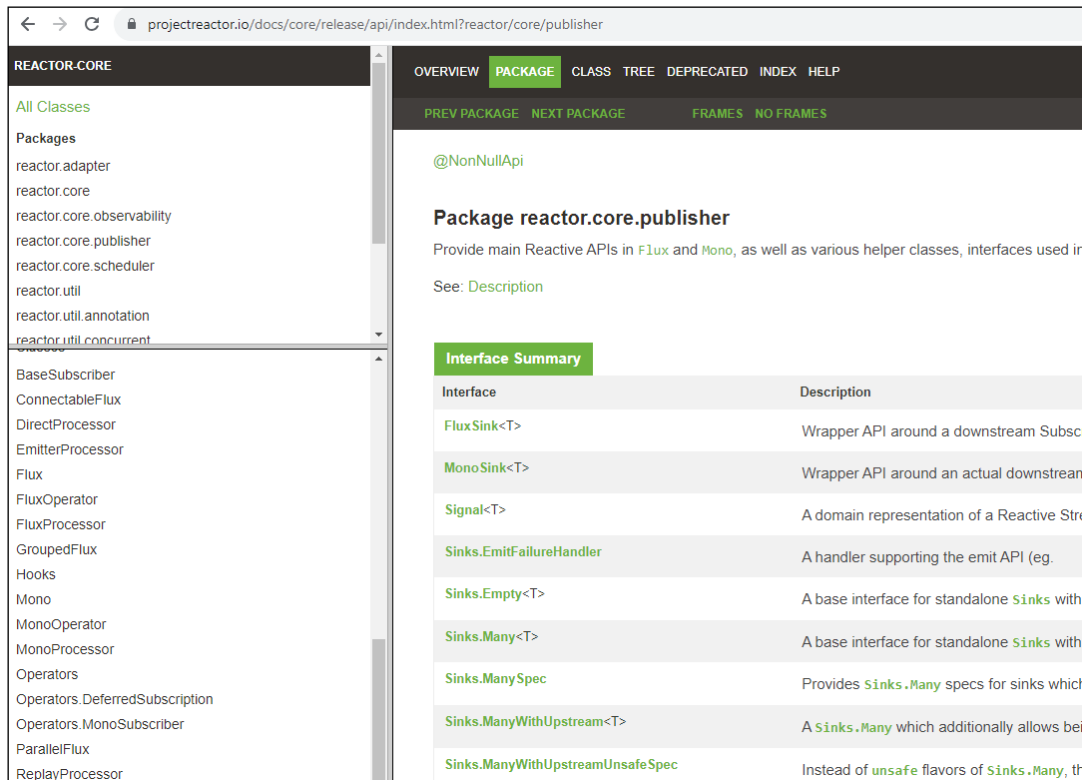
Flux y Mono

- Componentes principales del proyecto reactor
- Flux: Publicador en forma de Stream reactivo, que puede emitir de 0 a N elementos
- Mono: Publicador en forma de Stream reactivo, que puede emitir de 0 o 1 elemento
- Ambos implementan la interfaz Publisher
- Admiten numerosas operaciones de transformación durante su procesamiento



Ayuda oficial

<https://projectreactor.io/docs/core/release/api/index.html?reactor/core/publisher>



The screenshot displays the Project Reactor official API documentation website. The browser address bar shows the URL: `projectreactor.io/docs/core/release/api/index.html?reactor/core/publisher`. The page layout includes a sidebar on the left with a tree view of the API structure, and a main content area on the right.

REACTOR-CORE

- All Classes
- Packages
 - reactor.adapter
 - reactor.core
 - reactor.core.observability
 - reactor.core.publisher
 - reactor.core.scheduler
 - reactor.util
 - reactor.util.annotation
 - reactor.util.concurrent

reactor.core.publisher

- BaseSubscriber
- ConnectableFlux
- DirectProcessor
- EmitterProcessor
- Flux
- FluxOperator
- FluxProcessor
- GroupedFlux
- Hooks
- Mono
- MonoOperator
- MonoProcessor
- Operators
- Operators.DeferredSubscription
- Operators.MonoSubscriber
- ParallelFlux
- ReplayProcessor

PACKAGE

OVERVIEW **PACKAGE** CLASS TREE DEPRECATED INDEX HELP

PREV PACKAGE NEXT PACKAGE FRAMES NO FRAMES

@NonNullApi

Package reactor.core.publisher

Provide main Reactive APIs in `Flux` and `Mono`, as well as various helper classes, interfaces used in

See: [Description](#)

Interface Summary

Interface	Description
<code>FluxSink<T></code>	Wrapper API around a downstream Subscriber
<code>MonoSink<T></code>	Wrapper API around an actual downstream Subscriber
<code>Signal<T></code>	A domain representation of a Reactive Stream
<code>Sinks.EmitFailureHandler</code>	A handler supporting the emit API (eg. <code>Sinks.EmitFailureHandler.FAIL_FAST</code>)
<code>Sinks.Empty<T></code>	A base interface for standalone <code>Sinks</code> with no upstream
<code>Sinks.Many<T></code>	A base interface for standalone <code>Sinks</code> with an upstream
<code>Sinks.ManySpec</code>	Provides <code>Sinks.Many</code> specs for sinks which can be created from a <code>Spec</code>
<code>Sinks.ManyWithUpstream<T></code>	A <code>Sinks.Many</code> which additionally allows being created from an upstream
<code>Sinks.ManyWithUpstreamUnsafeSpec</code>	Instead of <code>unsafe</code> flavors of <code>Sinks.Many</code> , this interface provides a way to create a <code>Sinks.Many</code> from an upstream

Controlador WebFlux

➤ Un controlador para servicios reactivos basados en WebFlux genera flujos Flux y Mono a los que el cliente se puede suscribir:

```
@RestController
public class TestController {
    @GetMapping(value="names")
    public Flux<String> getNames(){
        ....
    }
    @GetMapping(value="name")
    public Mono<String> getName(){
        ....
    }
}
```



Creación objetos Flux

➤ La clase `reactor.core.publisher.Flux` dispone de varios métodos estáticos para crear un objeto Flux:

- `Flux<T> just(T...)`. Crea un objeto Flux a partir de una serie discreta de datos
 - `Flux<T> fromIterable(Iterable<T> it)`. Crea un objeto Flux a partir de un Iterable (lista, conjunto,...)
 - `Flux<T> fromStream(Stream<T> st)`. Crea un objeto Flux a partir de un Stream
 - `Flux<T> merge(Publisher<T> ... sources)`. Crea un objeto Flux fusionando una serie de objetos Publisher
- 

Métodos de Flux

➤ La clase Flux proporciona distintos métodos para manipular un flujo:

- **Flux<T> delayElements(Duration duration).** Genera un retardo en la entrega de cada elemento del flujo
- **Flux<T> filter(Predicate<T> pred).** Genera un flujo resultante de filtrar el flujo principal
- **Flux<V> map(Function<T,V> fun).** Transforma cada elemento emitido por el Flux en otro, aplicando una función de manera síncrona
- **Flux<V> flatMap(Function<T,Publisher<V>> fun).** Transforma cada elemento emitido por el Flux en un Publisher, aplicando una función de manera asíncrona.

Métodos de Flux (II)

- **Flux<T> switchIfEmpty(Publisher<T> pub).** Devuelve el flujo pasado como parámetro en caso de que el principal no emita elementos
- **Mono<void> then().** Devuelve un Mono vacío cuando el flujo finaliza. Existe una variante del método al que se le proporciona un Mono alternativo de devolución
- **Stream toStream().** Transforma el flujo en un Stream
- **Iterable toIterable().** Transforma el flujo en un Iterable
- **Mono<T> next() y last().** Emiten, respectivamente, el primer y último elemento del flujo como un Mono



Creación objeto Mono

➤ La clase `reactor.core.publisher.Mono` dispone de varios métodos estáticos para crear un objeto Mono:

- `Mono<T> just(T dato)`. Crea un objeto Mono a partir de un determinado elemento
- `Mono<T> fromSupplier(Supplier<T> sp)`. Crea un objeto Mono a partir del dato generado por el Supplier
- `Mono empty()`. Genera un Mono vacío



Métodos de Mono

➤ La clase Mono proporciona distintos métodos para manipular un elemento:

- `Mono<T> delayElement(Duration duration)`. Genera un retardo antes de emitir el elemento
- `Mono<R> map(Function<T,R> fun)`. Transforma el elemento emitido por el Mono en un Mono de otro elemento generado por la función de forma síncrona
- `Mono<R> flatMap(Function<T, Mono<R>> fun)`. Transforma el elemento emitido por el Mono en otro Mono generado por la función de forma asíncrona



Métodos de Mono II

- `Mono<V> then(Mono<V> other)`. Cuando el Mono completa, emite el Mono indicado como parámetro
- `Mono<Void> then()`. En este caso, devuelve un Mono vacío una vez que se complete
- `Mono<V> thenReturn(V element)`. Transforma el elemento emitido en otro, que es a su vez emitido en un nuevo Mono.
- `Mono<T> switchIfEmpty(Mono<T> other)`. Si el Mono principal está vacío al completarse, emite el mono alternativo
- `Mono<T> onErrorReturn(T fallback)`. Si se produce un error en el Mono, se emite un nuevo Mono con un valor especificado

Suscripción a flujo

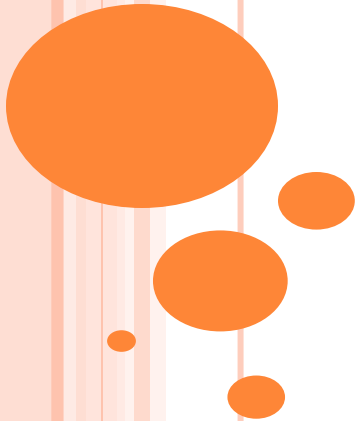
- El cliente se suscribe a los streams para procesar los datos según se van recibiendo.

```
Flux<String> flujo=objetoflux;  
flujo.subscribe(//implementación de consumer);
```

- Un cliente Java puede suscribirse a varios microservicios de forma asíncrona, y componer los resultados en un stream único

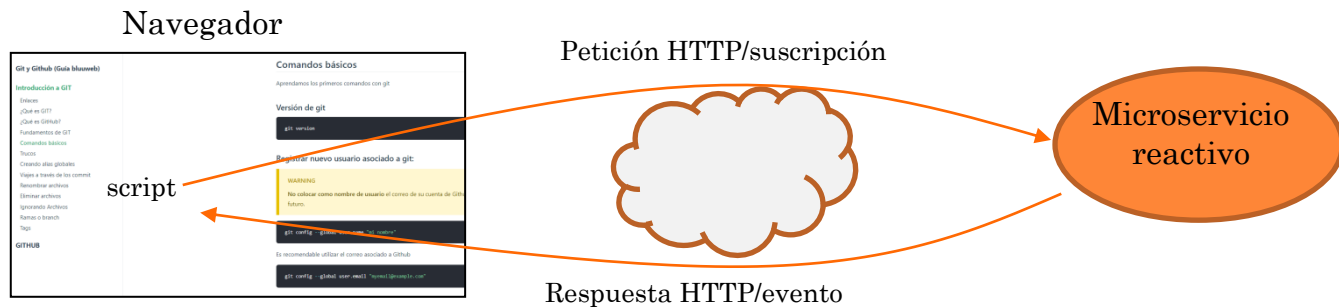


CONSUMIENDO SERVICIOS REACTIVOS



Tipos aplicaciones clientes I

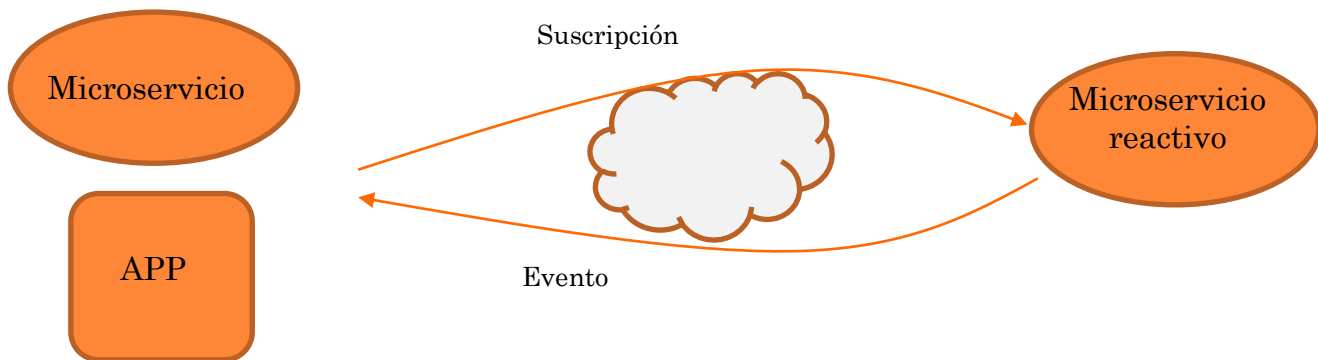
➤ **Aplicación front: Cliente final front basado en JavaScript, JQuery, Angular,..**



Tipos aplicaciones clientes II

➤ Aplicación independiente:

- Aplicación Web Spring
- Microservicio Spring Boot
- Otra aplicación



Cliente front JavaScript

➤ Objeto EventSource:

- Objeto JavaScript que permite establecer conexiones con un servidor y suscribirse a eventos generados por el mismo a través de un flujo de datos
- Únicamente para conexiones HTTP de tipo Get
- Permite definir funciones para responder a los eventos:
 - open: Conexión establecida
 - message: Llegada de datos
 - close: Cierre de la conexión



Cliente Java-Spring

➤ Para suscribirse a un flujo reactivo desde un cliente Spring utilizamos la interfaz WebClient.

➤ Forma parte del módulo Spring Web

suscripción al flujo

```
String url="http://localhost:8080/names";  
//creamos objeto WebClient  
WebClient webClient=WebClient.create(url);  
//petición al servicio reactivo  
Flux<Accion> datos=webClient  
    .get()  
    .accept(MediaType.TEXT_EVENT_STREAM)  
    .retrieve()  
    .bodyToFlux(String.class);  
datos.subscribe(System.out::println);  
: //otras tareas cliente
```



Métodos de suscripción

➤ Una vez obtenido el Flux/Mono, el cliente puede suscribirse al mismo a través de los siguientes métodos incluidos en dichas clases:

- `subscribe(Consumer<T> consumer)`. Se suscribe al Flux/Mono y ejecuta el consumer con cada dato que llega. El consumer no se ejecuta si el Mono o Flux no emiten elementos
- `doOnTerminate(Runnable run)`. Se ejecuta al completarse el Flux/Mono por cualquier razón, emitan o no datos
- `doOnError(Throwable t)`. Se ejecuta si el Flux/Mono termina por un error



Cliente RxJava

➤ Utilizamos el API `HttpClient` de Java SE para lanzar la petición al servicio remoto, convirtiendo el cuerpo de la respuesta en un `Flowable` u `Observable`:

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("http://localhost:8080/test"))
    .GET()
    .build();
CompletableFuture<HttpResponse<Stream<String>>> future =
    client.sendAsync(request, HttpResponse.BodyHandlers.ofLines());
// Creamos Observable a partir del future
Observable.fromFuture(future)
    // Convertimos la respuesta a un Observable<String>
    .flatMap(resp -> Observable.fromStream(resp.body()))
    .subscribe(
        msg -> System.out.println("Recibido: " + msg),
        err -> System.err.println("Error: " + err),
        () -> System.out.println("Stream completado")
    );
```

Si el servicio genera un `Flux<String>` de esta forma lo recogemos como `Stream<String>`

